

Exceptions

Errors

There are *two types* of errors in Python:

1. **Syntax errors:** These are errors in the *structure* of your code,
 - missing parentheses,
 - incorrect indentation, or misspelled keywords.

They prevent your code from running at all.

But they are also the *easiest* to fix

Because usually, the error message will tell you exactly what is wrong and where it is.

```
1 while True print('Hello world')
```

Exceptions

Exceptions are errors that occur *during* the execution of your code.

Even if a program is *syntactically correct*, it can still *raise* exceptions if it encounters an unexpected situation.

Not **necessarily** fatal, but something that you need to handle.

```
1  10 * (1/0)
2
3  '2' + 2
4
5  spam * 3
```

Exceptions

Exceptions come in different *types*, such as `ZeroDivisionError`, `TypeError`, and `NameError`.

There are *many* built-in exceptions in Python, and you can also *define your own* custom exceptions.

When an exception is raised, it *interrupts* the normal flow of your program, and if it is not handled, it will cause your program to crash.

Handling Exceptions

Sometimes, we *know* that our program will break under certain conditions

```
1 choice = int(input('Enter a number: '))
2
3 if choice == 1:
4     print('You entered 1')
5 else:
6     print('You did not enter 1')
```

We want to be able to *define* what happens when those conditions are met, instead of just crashing.

Handling Exceptions

In Python, we can handle exceptions using `try` and `except` blocks.

```
1  try:
2      choice = int(input('Enter a number: '))
3  except ValueError:
4      print('That is not a valid number!')
5
6  if choice == 1:
7      print('You entered 1')
8  else:
9      print('You did not enter 1')
```

`try` block *contains* the code that might raise an exception,

And the `except` block *contains* the code that will be executed if a specific exception is raised.

Think of it as an `if` statement

`if` *no problems*, just run the code, but `if` *there is a problem*, run the code in the `except` block instead.

Handling Exceptions

This is usually called a *try-catch* block, because you are *trying to run some code*, and if it fails, you *catch the exception and handle it*.

```
1  try:
2      choice = int(input('Enter a number: '))
3      print(choice)
4  except ValueError:
5      print('That is not a valid number!')
6  print(choice+1)
```

This works as follows:

1. Python sees the `try` block, and starts executing the code inside it
2. the code inside the `try` block is run
3. if *no exceptions* are raised, the `except` block is skipped
4. if *an exception is raised*, the rest of the code in `try` is skipped
5. Python *looks for* an `except` block that matches the type of the exception, then runs that
6. if *no matching* `except` block is found, the exception is raised again, usually leading to a crash

Exceptions

Exceptions are **classes**. Bundles of data and functions that represent a specific type of error.

When an exception is raised, an *instance* of the exception class is created, and it contains information about the error that occurred.

You can access this information using the `as` keyword in the `except` block.

```
1  try:
2      choice = int(input('Enter a number: '))
3  except ValueError as e:
4      print('That is not a valid number!')
5      print(e)
```


Exceptions

If you don't know what *specific type* of exception might be raised, you can use a *general* `except` block that will catch *any* exception.

```
1  try:
2      choice = int(input('Enter a number: '))
3  except Exception as e:
4      print('An error occurred!')
5      print(e)
```

or, if you don't care about the exception details, you can just use a bare `except` block

```
1  try:
2      choice = int(input('Enter a number: '))
3  except:
4      print('An error occurred!')
```

Creating Custom Exceptions

Exceptions are *just classes*, so you can create your own custom exceptions by defining a new class that *inherits* from the built-in `Exception` class.

```
1 class InvalidMove(Exception):
2     pass
3
4 if int(input('Enter a number: ')) > 4:
5     raise InvalidMove('That move is not allowed!')
```

Creating Custom Exceptions

The beauty of doing this is that you can *add additional information* to your custom exception class, and then access that information when the exception is raised.

```
1 class InvalidMove(Exception):
2     def __init__(self, message, move):
3         super().__init__(message)
4         self.move = move
```

This allows you to *provide more context* about the error that occurred, which can be very helpful for debugging and handling the exception appropriately.

Especially in larger teams and in larger programs

And since this is *also* just an exception, you can *handle* it using a `try-except` block just like any other exception.

Why make custom exceptions?

The golden rule for exceptions is that you should *only* catch exceptions that you can *handle* appropriately,

And let the rest of the exceptions *propagate up*, so that they can be handled *somewhere else*, or by the default exception handler.

If an *actual* unexpected error occurs, you should let it crash your program, so that you can *fix the underlying issue* instead of just *hiding the symptoms*.

Handle **all** exceptions you *know* will happen

Addition to the project

In your OOP game project, add at least *one* custom exception, and *handle* it appropriately in your code.

For example, you could create a custom exception called `InvalidMove`, and raise it whenever *the player tries to make an invalid move* in the game.

Then, you can handle this exception in your game loop, and *provide feedback* to the player about why their move was invalid, and what they can do to fix it.

Any time you ask for user Input usually has the potential to raise an exception

i.e. player makes a move that is not allowed, a choice that doesn't exist, etc.